

1. Цикли

Циклите позволяват да се извършва някакво действие в зависимост от изпълнението на някакво условие. Python има следните типове цикли:

- **while**
- **for**

1.1. Цикъл while

Цикълът **while** тества истинността на някакво условие и ако условието е вярно, се изпълняват изразите на цикъла. Има следното формално определение:

while условен_израз:
инструкции

Ключовата дума **while** е последвана от условен израз и докато този израз връща стойност True, блокът от оператори, който следва, ще бъде изпълнен. Всички оператори, които принадлежат към цикъла while, са разположени на следващите редове и трябва да бъдат с отстъп от началото на ключовата дума while.

```
number = 1
```

```
while number < 5:
```

```
    print(f"number = {number}")
```

```
    number += 1
```

```
print("Програмата е прекратена")
```

main.py	Run	Shell
<pre>1 number = 1 2 while number < 5: 3 print(f"number = {number}") 4 number += 1 5 print("Програмата е прекратена")</pre>		<pre>number = 1 number = 2 number = 3 number = 4 Програмата е прекратена</pre>

В този случай цикълът `while` ще работи, докато променливата `number` е по-малка от 5. Самият блок на цикъла се състои от две инструкции:

```
print(f"number = {number}")  
number += 1
```

Тези инструкции са с отстъп от началото на израза `while` - в този случай от началото на реда. Това позволява на Python да определи дали те принадлежат на цикъла. В самия цикъл първо се показва стойността на числовата променлива и след това ѝ се присвоява нова стойност. Така също трябва да се има предвид, че последният оператор `print("Програмата е прекратена")` не е с отстъп от началото на реда, така че не е включен в цикъла `while`.

Целият процес на цикъла може да бъде представен по следния начин:

1. Първо се проверява стойността на числовата променлива - по-голяма ли е от 5. И тъй като променливата първоначално е равна на 1, това условие връща `True`, и следователно се изпълняват инструкциите на цикъла. Инструкциите за цикъл извеждат низ към конзолата `number = 1`. И тогава стойността на променливата т.е. `number` се увеличава с 1 т.е. `number = 2`. Еднократното изпълнение на блок от инструкции за цикъл се нарича итерация. Тоест по този начин първата итерация се изпълнява в цикъла.

2. Състоянието се проверява отново $number < 5$. Все още е вярно, защото числото = 2, така че операторите на цикъла се изпълняват. Инструкциите за цикъл извеждат низ към конзолата $number = 2$. И тогава стойността на променливата $number$ отново се увеличава с 1 т.е. $number = 3$. По този начин се извършва втората итерация.
3. Състоянието се проверява отново $number < 5$. Все още е вярно, защото числото = 3, така че операторите на цикъла се изпълняват. Инструкциите за цикъл извеждат низ към конзолата $number = 3$. И тогава стойността на променливата $number$ отново се увеличава с едно - сега е равна на 4. Тоест се извършва третата итерация.
4. Състоянието се проверява отново $number < 5$. Все още е вярно, защото числото = 4, така че операторите на цикъла се изпълняват. Инструкциите за цикъл извеждат низ към конзолата $number = 4$. И тогава стойността на числовата променлива отново се увеличава с едно - сега е равна на 5. Тоест се извършва четвъртата итерация.
5. И състоянието се проверява отново $number < 5$. Но сега е **False**, защото числото = 5, така че има изход от цикълът. Целият цикъл е завършен. Освен това действията, които са дефинирани след цикъла, вече са изпълнени. Така че този цикъл ще направи четири преминавания или четири итерации. В резултат на това, когато се изпълнява кода, ще се получи следния конзолен изход:

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

```
number = 1
number = 2
number = 3
number = 4
number = 5. Цикълът завърши
Програмата приключи
```

main.py	Run	Shell
<pre>1 number = 1 2 while number < 5: 3 print(f"number = { number }") 4 number += 1 5 else: 6 print(f"number = {number}. Цикълът завърши") 7 print("Програмата е прекратена")</pre>		<pre>number = 1 number = 2 number = 3 number = 4 number = 5. Цикълът завърши Програмата е прекратена ></pre>

Блокът else може да бъде полезен, ако условието първоначално е False и може да се направи нещо по въпроса:

```
number = 10
while number < 5:
    print(f" number = { number }")
    number += 1
else:
    print(f"number = {number}. Цикълът завърши")
print("Програмата е прекратена")
```

main.py	Run	Shell
<pre>1 number = 10 2 while number < 5: 3 print(f" number = { number }") 4 number += 1 5 else: 6 print(f"number = {number}. Цикълът завърши") 7 print("Програмата е прекратена")</pre>		<pre>number = 10. Цикълът завърши Програмата е прекратена ></pre>

В този случай условието `number < 5` първоначално е False, така че цикълът не се повтаря и прескача направо към else блока.

2. Цикъл for

Друг тип цикъл е конструкцията **for**. Този цикъл повтаря набор от стойности, поставя всяка стойност в променлива и след това в цикъла може да се извършват различни действия с тази променлива. Формалната дефиниция на цикъл **for** е:

```
for стойност in набор_от_стойности:  
    инструкции
```

Ключовата дума **for** е последвана от името на променливата, в която ще бъдат поставени стойностите. След това операторът **in** е последван от набор от стойности и двоеточие. И от следващия ред има блок с инструкции за цикъл, който също трябва да бъде с отстъп от началото на цикъла. Когато изпълнява цикъл, Python последователно получава всички стойности от набора и ги предава на променливата. Когато всички стойности в набора бъдат повторени, цикълът приключва. Като набор от стойности, например, може да е низ, който всъщност представлява набор от знаци.

Нека да се разгледаме примера:

```
message = "Hello"  
for c in message:  
    print(c)
```

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Цикълът дефинира променливата **c**, след оператора **in**, променливата за съобщението се посочва като повторен набор, който съхранява низа "Hello". В резултат на това цикълът **for** ще премине през всички знаци от низа на съобщението последователно и ще ги постави в променлива **c**. Самият блок на цикъла се състои от един оператор, който отпечатва стойността на променливата **c** в конзолата. Конзолен изход на програмата:

Н
е
л
л
о

main.py		Run	Shell
1	message = "Hello"		H
2	for c in message:		e
3	print(c)		l
4			l
5			o
			>

Цикълът **for** може също да има допълнителен блок **else**, който се изпълнява след края на цикъла:

```
message = "Hello"
```

```
for c in message:
```

```
    print(c)
```

```
else:
```

```
    print(f"Последен символ: {c}. Цикълът завършва");
```

```
print("Програмата приключи")
```

```
# инструкцията няма достъп, затова не се отнася до else
```

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

В този случай се получава следния конзолен изход:

Н
е
л
л
о

main.py	Run	Shell
<pre>1 message = "Hello" 2 for c in message: 3 print(c) 4 else: 5 print(f"Последен символ: {c}. Цикълът завършва"); 6 print("Програмата приключи") 7 # инструкцията няма достъп, затова не се отнася до else</pre>		<pre>Н е л л о Последен символ: о. Цикълът завършва Програмата приключи</pre>

Последния символ: о. Цикълът завършва. Програмата приключи

Трябва да се отбележи, че блокът `else` има достъп до всички променливи, които са дефинирани в цикъла `for`.

3. Вложени цикли

Някои цикли в себе си могат да съдържат други цикли.

Пример: Да се създаде програма, чийто изход е таблицата за умножение:

```
i = 1
j = 1
while i < 10:
    while j < 10:
        print(i * j, end="t")
        j += 1
    print("\n")
    j = 1
    i += 1
```

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Външният цикъл `while i < 10:` се изпълнява 9 пъти, докато променливата `i` стане равна на 10. Вътре в този цикъл се изпълнява вътрешният цикъл `while j < 10:`. Вътрешният цикъл също се изпълнява 9 пъти, докато променливата `j` стане равна на 10. Освен това всички 9 итерации на вътрешния цикъл се задействат в рамките на една итерация на външния цикъл. При всяка итерация на вътрешния цикъл, произведението на числата `i` и `j` ще се показват в конзолата. Тогава стойността на променливата `j` се увеличава с единица. Когато вътрешният цикъл приключи работата си, стойността на променливата `j` се нулира на 1, а стойността на променливата `i` се увеличава с единица и се извършва преходът към следващата итерация на външния цикъл. И всичко се повтаря, докато променливата `i` стане равна на 10. Съответно вътрешният цикъл ще работи само 81 пъти за всички итерации на външния цикъл. В резултат на това ще получим следния конзолен изход:

main.py

Run

```
1 i = 1
2 j = 1
3 while i < 10:
4     while j < 10:
5         print(i * j, end="\t")
6         j += 1
7     print("\n")
8     j = 1
9     i += 1
10
```

Shell

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Вложените for цикли могат да бъдат дефинирани по подобен начин:

```
for c1 in "ab":  
    for c2 in "ba":  
        print(f"{c1}{c2}")
```

В този случай външният цикъл преминава през низа "ab" и поставя всеки знак в променливата **c1**. Вътрешният цикъл итерира над низа "ba", поставя всеки знак в низа в променлива **c2** и извежда комбинацията от двата знака в конзолата. Т.е. в крайна сметка се получават всички възможни комбинации от знаци a и b:

ab
aa
bb
ba

main.py				Run	Shell
1	for c1 in "ab":				ab
2	for c2 in "ba":				aa
3	print(f"{c1}{c2}")				bb
4					ba

4. Изход от цикъл, прекъсване и продължаване

За да се контролира цикъла, могат да се използват специалните оператори `break` и `continue`. Операторът `break` излиза от цикъла. А операторът `continue` прескача към следващата итерация на цикъла. Операторът `break` може да се използва, ако в цикъла се формират условия, които са несъвместими с по-нататъшното му изпълнение.

Пример:

```
number = 0
while number < 5:
    number += 1
    if number == 3 :    # ако number = 3, изход от цикла
        break
    print(f"number = {number}")
```

main.py				Shell
<pre>1 number = 0 2 while number < 5: 3 number += 1 4 if number == 3 : # ако number = 3, изход от цикла 5 break 6 print(f"number = {number}")</pre>				<pre>number = 1 number = 2 > </pre>

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Тук цикълът `while` проверява условието `number < 5`. И докато числото не е равно на 5, стойността на числото трябва да бъде отпечатана на конзолата. Въпреки това, друго условие също се проверява вътре в цикъла: `if number == 3`. Тоест, ако стойността на числото е 3, тогава с помощта на оператора `break` се излиза от цикъла. И накрая се получава следния конзолен изход:

```
number = 1
number = 2
```

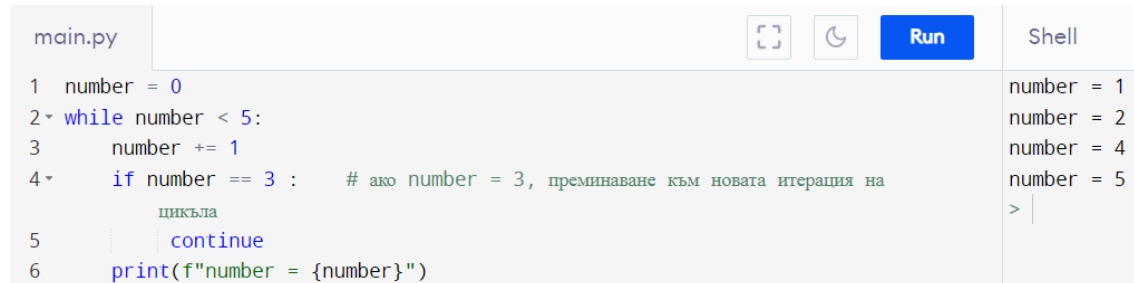
За разлика от оператора `break`, операторът за продължаване прескача към следващата итерация на цикъла, без да го прекратява. Ако в предишния пример се направи замяна на `break` с `continue`:

```
number = 0
while number < 5:
    number += 1
    if number == 3 :    # ако number = 3, преминаване към новата итерация на цикъла
        continue
    print(f"number = {number}")
```

То в този случай, ако стойността на числовата променлива е 3 и резултатът ще бъде:

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

```
number = 1  
number = 2  
number = 4  
number = 5
```



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 number = 0  
2 while number < 5:  
3     number += 1  
4     if number == 3:      # ако number = 3, преминаване към новата итерация на  
                           цикъла  
5         continue  
6     print(f"number = {number}")
```

On the right side, there is a 'Shell' window showing the output of the program:

```
number = 1  
number = 2  
number = 4  
number = 5  
>
```

5. Функции

Функциите, които представляват блок код, който изпълнява определена задача и може да се използва повторно в други частни програми. До момента бяха разгледани и използвани няколко функции: като функцията `print()`, която извежда някаква стойност на конзолата. Python разполага с множество вградени функции, като позволява да дефинира и свои собствени. Формално определение за функции е:

def име_на_функция ([параметри]):
инструкции

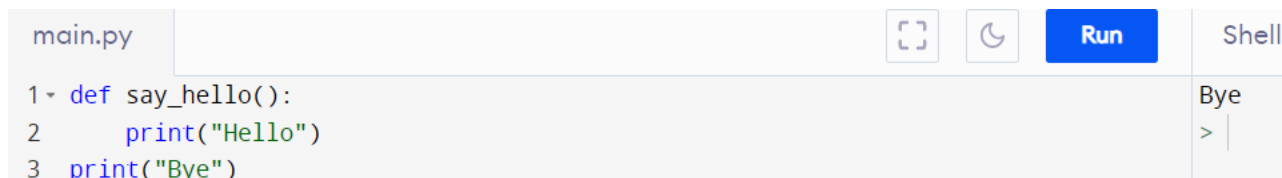
Дефиницията на функцията започва с израз `def`, който се състои от името на функцията, набор от скоби с параметри и двоеточие. Параметрите в скоби не са задължителни. И от следващия ред идва блок от инструкции, които функцията изпълнява. Всички функционални инструкции са с отстъп от началото на реда. Например дефиницията на най-простата функция:

```
def say_hello():  
    print("Hello")
```

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Функцията се нарича **say_hello**. Тя няма параметри и съдържа един-единствен израз, който отпечатва низа "Hello" на конзолата. Трябва да се знае, че операторите на функцията трябва да са с отстъп от началото на функцията. Например:

```
def say_hello():  
    print("Hello")  
print("Bye")
```



```
main.py  
1 def say_hello():  
2     print("Hello")  
3     print("Bye")
```

Тук инструкцията `print("Bye")` не е с отстъп от началото на функцията `say_hello` и следователно не е включена в тази функция. Обикновено има два празни реда между дефиницията на функцията и останалите инструкции, които не са включени във функцията. За извикване на функция се посочва името на функцията, след което стойностите за всички нейни параметри се предават в скоби:

име_на_функция ([параметри])

Например, нека се дефинира и извика функцията:

```
def say_hello():          # дефиниция на функцията say_hello  
    print("Hello")  
  
say_hello()               # извикване на функцията say_hello  
say_hello()  
say_hello()
```

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Тук функцията `say_hello` се извиква три пъти подред. В резултат на това ще се получи следния конзолен

изход:

Hello
Hello
Hello

```
main.py
1 def say_hello():           # дефиниция на функцията say_hello
2     print("Hello")
3 say_hello()                # извикване на функцията say_hello
4 say_hello()
5 say_hello()

Hello
Hello
Hello
> |
```

Трябва да се има предвид, че функцията първо се дефинира и след това се извиква. Ако функцията има един израз, тогава тя може да бъде поставена на същия ред като останалата част от дефиницията на функцията:

```
def say_hello(): print("Hello")
say_hello()
```

```
main.py
1 def say_hello(): print("Hello")
2 say_hello()

Hello
> |
```

Други функции могат да бъдат дефинирани и извикани по същия начин. Например, нека се дефинират и изпълнят няколко функции:

```
def say_hello():
    print("Hello")
def say_goodbye():
    print("Good Bye")
say_hello()
say_goodbye()
```

```
main.py
1 def say_hello():
2     print("Hello")
3 def say_goodbye():
4     print("Good Bye")
5 say_hello()
6 say_goodbye()

Hello
Good Bye
> |
```

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Изход на конзолата:

Hello

Good Bye

6. Локални функции

Някои функции могат да бъдат дефинирани вътре в други функции - вътрешните функции се наричат също локални. Локалните функции могат да се използват само в рамките на функцията, в която са дефинирани. Например:

```
def print_messages():
```

```
    # дефиниция на локални функции
```

```
    def say_hello(): print("Hello")
```

```
    def say_goodbye(): print("Good Bye")
```

```
    # извикване на локални функции
```

```
    say_hello()
```

```
    say_goodbye()
```

```
# Извикване на функцията print_messages
```

```
print_messages()
```

```
#say_hello() # Извън функцията print_messages say_hello не е налична
```

Тук функциите `say_hello()` и `say_goodbye()` са дефинирани в рамките на функцията `print_messages()` и следователно са локални спрямо нея. Съответно те могат да се използват само вътре във функция `print_messages()`.

```
main.py
1- def print_messages():
2     # дефиниция на локални функции
3     def say_hello(): print("Hello")
4     def say_goodbye(): print("Good Bye")
5     # извикване на локални функции
6     say_hello()
7     say_goodbye() # Извикване на функцията print_messages
8     print_messages()
9     #say_hello() # извън функцията print_messages say_hello не е налична
```

Shell

```
Hello
Good Bye
>
```

7. Програмна организация и основна функция

Много функции могат да бъдат дефинирани в една програма. И за да се организират всички, един от начините да се организират е да се добави специална функция (обикновено наричана `main`), в която след това се извикват други функции:

```
def main():
```

```
    say_hello()
```

```
    say_goodbye()
```

```
def say_hello():
```

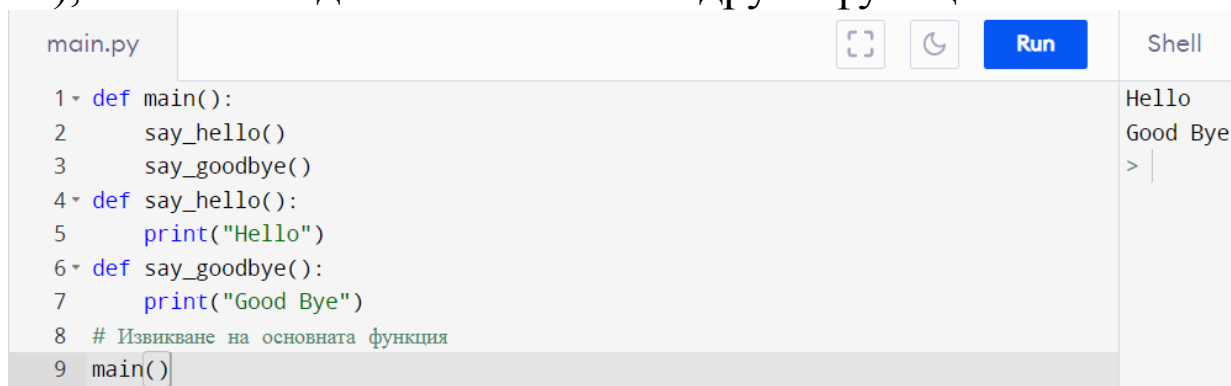
```
    print("Hello")
```

```
def say_goodbye():
```

```
    print("Good Bye")
```

```
# Извикване на основната функция
```

```
main()
```



```
main.py

1 def main():
2     say_hello()
3     say_goodbye()
4 def say_hello():
5     print("Hello")
6 def say_goodbye():
7     print("Good Bye")
8 # Извикване на основната функция
9 main()
```

Shell

```
Hello
Good Bye
>
```

8. Функционални параметри

Функцията може да приема параметри. Може да се предават данни към функция чрез параметри. Често срещан пример е функцията `print()`, която, като използва параметър, приема стойност, която се отпечатва на конзолата. Сега нека да се дефинира и използва конкретната функция с параметри:

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

```
def say_hello(name):  
    print(f'Hello, {name}')
```

```
say_hello("Tom")  
say_hello("Bob")  
say_hello("Alice")
```

Функцията `say_hello` има параметър `name` и когато функцията бъде извикана, може да се предаде някаква стойност към този параметър. Вътре във функцията може да се използва параметъра, като обикновена променлива, например да се отпечата стойността на този параметър в конзолата с функцията за печат. И така, в изрази:

```
say_hello("Tom")
```

Низът "Tom" ще бъде предаден на параметъра `name`. В резултат на това при изпълнение на програмата ще се получи следния конзолен изход:

```
Hello Tom  
Hello Bob  
Hello Alice
```

main.py			Run	Shell
<pre>1- def say_hello(name): 2 print(f'Hello, {name}')</pre>				<pre>Hello, Tom Hello, Bob Hello, Alice > </pre>

Когато функцията се извика, стойностите се предават на параметрите по позиция. Например, ако се дефинира и извика функция с няколко параметъра:

```
print_person(name, age):  
    print(f'Name: {name}')
```

```
    print(f'Age: {age}')
```

```
print person("Tom", 37)
```

main.py			Run	Shell
<pre>1- def print_person(name, age): 2 print(f'Name: {name}')</pre>				<pre>Name: Tom Age: 37 > </pre>

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Където функцията `print_person` приема два параметъра: име и възраст. При извикване на функцията:

```
print_person("Tom", 37)
```

Първата стойност - "Tom" се предава на първия параметър, т.е. `name`, а втората стойност - 37 се предава на втория параметър, т.е. `age`, като във функцията, стойностите на параметрите се отпечатват на конзолата:

```
Name: Tom
```

```
Age: 37
```

9. Стойности по подразбиране

Могат да се направят някои параметри на функцията незадължителни, като се посочат стойностите по подразбиране за тях, когато се дефинира функцията. Например:

```
def say_hello(name="Tom"):
```

```
    print(f"Hello, {name}")
```

```
say_hello()          # параметъра name приема стойност "Tom"
```

```
say_hello("Bob")     # name = "Bob"
```

Тук параметърът `name` не е задължителен. И ако не предадем стойност за него при извикване на функцията, тогава се прилага стойността по подразбиране, т.е. низът "Tom". Конзолен изход на тази програма:

```
Hello Tom
```

```
Hello Bob
```

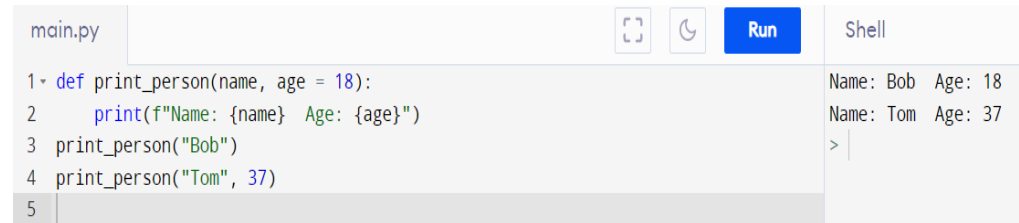
main.py	  	Shell
<pre>1 def say_hello(name="Tom"): 2 print(f"Hello, {name}") 3 say_hello() #параметъра name приема стойност "Tom" 4 say_hello("Bob") #name = "Bob"</pre>		<pre>Hello, Tom Hello, Bob > </pre>

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Ако функцията има множество параметри, незадължителните параметри трябва да са след задължителните.

Пример:

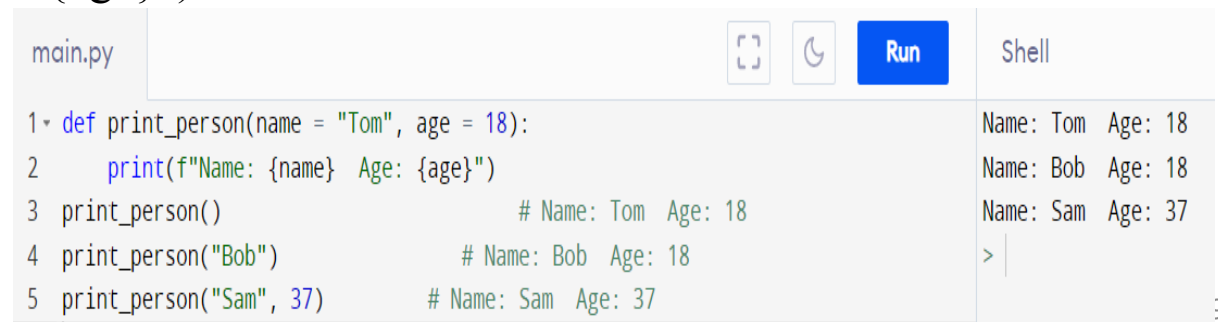
```
def print_person(name, age = 18):  
    print(f"Name: {name} Age: {age}")  
print_person("Bob")  
print_person("Tom", 37)
```



```
main.py  Run Shell  
1 def print_person(name, age = 18):  
2     print(f"Name: {name} Age: {age}")  
3 print_person("Bob")  
4 print_person("Tom", 37)  
5  
Name: Bob Age: 18  
Name: Tom Age: 37  
>
```

Тук параметърът age не е задължителен и по подразбиране е 18. Той е предшестван от необходимия параметър за име. Следователно, когато се извика функцията, не може да се предаде стойност на параметъра age, но е необходимо да се предаде стойност на параметъра name. Ако е необходимо, може да се направят всички параметри незадължителни:

```
def print_person(name = "Tom", age = 18):  
    print(f"Name: {name} Age: {age}")  
print_person()  
print_person("Bob")  
print_person("Sam", 37)
```



```
main.py  Run Shell  
1 def print_person(name = "Tom", age = 18):  
2     print(f"Name: {name} Age: {age}")  
3 print_person() # Name: Tom Age: 18  
4 print_person("Bob") # Name: Bob Age: 18  
5 print_person("Sam", 37) # Name: Sam Age: 37  
Name: Tom Age: 18  
Name: Bob Age: 18  
Name: Sam Age: 37  
>
```

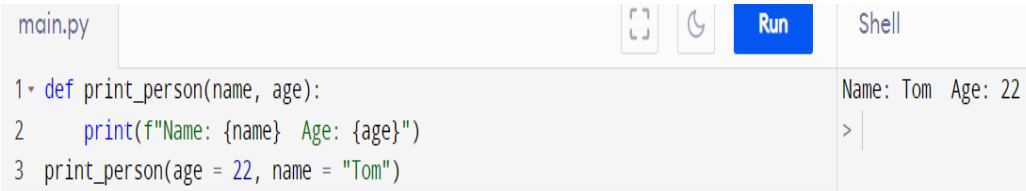
10. Предаване на стойности към параметри по име. Именувани параметри

В примерите по-горе, когато функцията се извиква, стойностите се предават като параметри на функцията по позиция. Също така е възможно да се предават стойности на параметри по име. За да се направи това, при извикване на функцията се посочва името на параметъра и се присвоява съответната стойност:

```
def print_person(name, age):
```

```
    print(f"Name: {name} Age: {age}")
```

```
print_person(age = 22, name = "Tom")
```



```
main.py  Run  Shell
1 def print_person(name, age):
2     print(f"Name: {name} Age: {age}")
3 print_person(age = 22, name = "Tom")
Name: Tom Age: 22
>
```

В този случай параметрите за възраст и име се предават по име. И въпреки факта, че параметърът `name` е на първо място в дефиницията на функцията, при извикване на функцията може да се запише `print_person(age = 22, name = "Tom")` и по този начин да се предаде числото 22 към параметъра `age`, а низът "Tom" към параметъра `name`. **Знакът `*` позволява да се зададат кои параметри ще бъдат именувани – т.е. параметри, на които могат да се предават стойности само по име. Всички параметри, които се намират вдясно от символа `*`, получават стойности само по име:**

```
def print_person(name, *, age, company):
```

```
    print(f"Name: {name} Age: {age} Company: {company}")
```

```
print_person("Bob", age = 41, company = "Microsoft")
```

```
# Name: Bob Age: 41 company: Microsoft
```

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

```
main.py
1- def print_person(name, *, age, company):
2-     print(f"Name: {name} Age: {age} Company: {company}")
3- print_person("Bob", age = 41, company = "Microsoft")
4- # Name: Bob Age: 41 company: Microsoft
```

Shell

```
Name: Bob Age: 41 Company: Microsoft
>
```

В този случай възрастта и параметрите на фирмата са наименувани. **Могат да бъдат наименувани всички параметри, като се постави префикс към списъка с параметри с *:**

```
def print_person(*, name, age, company):
```

```
    print(f"Name: {name} Age: {age} Company: {company}")
```

Ако, обаче, трябва да се дефинират параметри, на които могат да се предават стойности само по позиция, т.е. позиционни параметри, тогава може да се използва символа / : **всички параметри, които вървят преди символа / са позиционни и могат да получават само стойности по позиция:**

```
def print_person(name, /, age, company="Microsoft"):
```

```
    print(f"Name: {name} Age: {age} Company: {company}")
```

```
print_person("Tom", company="JetBrains", age = 24)
```

```
# Name: Tom Age: 24 company: JetBrains
```

```
print_person("Bob", 41)
```

```
# Name: Bob Age: 41 company: Microsoft
```

```
main.py
1- def print_person(name, /, age, company="Microsoft"):
2-     print(f"Name: {name} Age: {age} Company: {company}")
3- print_person("Tom", company="JetBrains", age = 24)
4- # Name: Tom Age: 24 company: JetBrains
5- print_person("Bob", 41)
6- # Name: Bob Age: 41 company: Microsoft
```

Shell

```
Name: Tom Age: 24 Company: JetBrains
Name: Bob Age: 41 Company: Microsoft
>
```

В този случай параметърът name е позиционен. **За една функция може да се дефинират едновременно позиционни и именувани параметри.**

```
def print_person(name, /, age = 18, *, company):
```

```
    print(f"Name: {name} Age: {age} Company: {company}")
```

```
print_person("Sam", company = "Google") # Name: Sam Age: 18 company: Google
```

```
print_person("Tom", 37, company = "JetBrains") # Name: Tom Age: 37 company: JetBrains
```

```
print_person("Bob", company = "Microsoft", age = 42) # Name: Bob Age: 42 company: Microsoft
```

main.py	<pre>1- def print_person(name, /, age = 18, *, company): 2 print(f"Name: {name} Age: {age} Company: {company}") 3 print_person("Sam", company = "Google") # Name: Sam Age: 18 company: Google 4 print_person("Tom", 37, company = "JetBrains") # Name: Tom Age: 37 company: JetBrains 5 print_person("Bob", company = "Microsoft", age = 42) # Name: Bob Age: 42 company: Microsoft</pre>	Shell
		<pre>Name: Sam Age: 18 Company: Google Name: Tom Age: 37 Company: JetBrains Name: Bob Age: 42 Company: Microsoft > </pre>

В този случай параметърът name се намира отляво на символа /, следователно е позиционен и задължителен и може да се предава само стойност по позиция. Параметърът company е наименуван, защото се намира вдясно от символа *. Параметърът age може да получи стойност по име и по позиция.

11. Неопределен брой параметри

Символът звездичка може да се използва за дефиниране на параметър, през който могат да се предават неопределен брой стойности. Това може да бъде полезно, когато трябва функция да получава множество стойности, но не е уточнен техният брой. Например, нека се дефинира функция за изчисляване на сумата от числа:

```
def sum(*numbers):
    result = 0
    for n in numbers:
        result += n
    print(f"sum = {result}")
sum(1, 2, 3, 4, 5)    # sum = 15
sum(3, 4, 5, 6)      # sum = 18
```

main.py	<pre>1- def sum(*numbers): 2 result = 0 3- for n in numbers: 4 result += n 5 print(f"sum = {result}") 6 sum(1, 2, 3, 4, 5) # sum = 15 7 sum(3, 4, 5, 6) # sum = 18</pre>	Shell
		<pre>sum = 15 sum = 18 > </pre>

В този случай функцията за сумиране приема един параметър т.е. *numbers, но звездичката пред името на параметъра показва, че всъщност може да се предаде неопределен брой стойности или набор от стойности на мястото на този параметър. В самата функция, като се използва цикъла for, може да се премине през този набор, да се получи всяка стойност от този набор в променлива n и да извърши някакво действие с него. Например, в този случай се изчислява сборът от прехвърлените числа.

12. Инструкцията return и връщане на резултат от функция

Връщане на резултат

Функцията може да върне резултат. За да направи това, функцията използва израза за връщане, след което се посочва върнатата стойност:

def име_на_функция([параметри]):

инструкции

return на_връщана_стойност

Нека се дефинира най-простата функция, която връща стойност:

def get_message():

return "Hello abv.bg"

Тук след оператора return има ред "Hello abv.bg" т.е. това е стойността и функцията ще върне get_message(). След това резултатът от тази функция може да бъде присвоен на променлива или използван като нормална стойност:

def get_message():

return "Hello abv.bg"

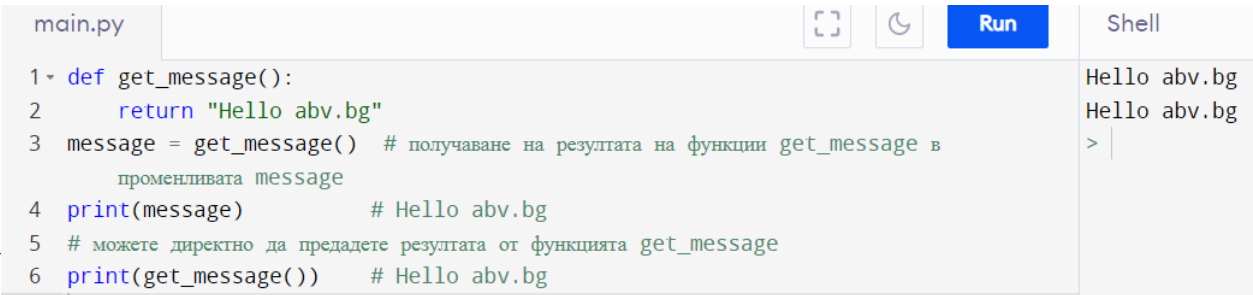
message = get_message()

променливата message

print(message) # Hello ab

можете директно да предадете резултата от функцията get_message

print(get_message()) # Hello abv.bg



```
main.py
1 def get_message():
2     return "Hello abv.bg"
3 message = get_message() # получаване на резултата на функции get_message в
                           променливата message
4 print(message)          # Hello abv.bg
5 # можете директно да предадете резултата от функцията get_message
6 print(get_message())    # Hello abv.bg
```

Run Shell

Hello abv.bg
Hello abv.bg
> |

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Инструкцията за връщане може също да бъде последвана от сложен изчислителен израз, чийто резултат ще бъде върнат от функцията. Например, нека се дефинира функция, която удвоява число:

```
def double(number):  
    return 2 * number
```

Тук функцията `double` ще върне резултата от израза `2 * number`:

```
def double(number):  
    return 2 * number  
result1 = double(4)  
result2 = double(5)  
print(f"result1 = {result1}")  
print(f"result2 = {result2}")
```

main.py	Run	Shell
<pre>1 def double(number): 2 return 2 * number 3 result1 = double(4) # result1 = 8 4 result2 = double(5) # result2 = 10 5 print(f"result1 = {result1}") # result1 = 8 6 print(f"result2 = {result2}") # result2 = 10</pre>		<pre>result1 = 8 result2 = 10 > </pre>

Или друг пример т.е. получаване на сбора от числа:

```
def sum(a, b):  
    return a + b  
result = sum(4, 6)  
print(f"sum(4, 6) = {result}")  
print(f"sum(3, 5) = {sum(3, 5)}")
```

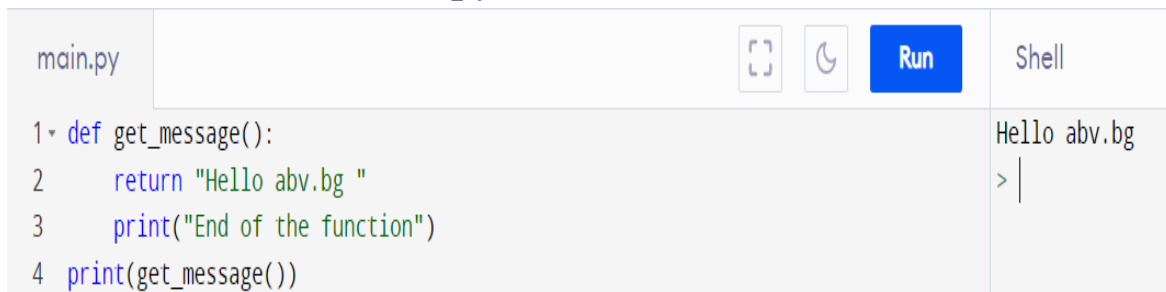
main.py	Run	Shell
<pre>1 def sum(a, b): 2 return a + b 3 result = sum(4, 6) # result = 0 4 print(f"sum(4, 6) = {result}") # sum(4, 6) = 10 5 print(f"sum(3, 5) = {sum(3, 5)}") # sum(3, 5) = 8</pre>		<pre>sum(4, 6) = 10 sum(3, 5) = 8 > </pre>

13. Излизане от функция

Операторът `return` не само връща стойност, но и излиза от функцията. Следователно, той трябва да бъде дефиниран след останалите инструкции.

Пример:

```
def get_message():  
    return "Hello abv.bg "  
    print("End of the function")  
print(get_message())
```



```
main.py  
1 def get_message():  
2     return "Hello abv.bg "  
3     print("End of the function")  
4 print(get_message())
```

Run Shell

Hello abv.bg
> |

От гледна точка на синтаксиса тази функция е правилна, но нейното изявление `print("End of the function")` няма смисъл и никога няма да бъде изпълнено, тъй като преди да бъде изпълнено, операторът `return` ще върне стойност и ще излезе от функцията. Въпреки това може да се използва израз `return` във функции, които не връщат никаква стойност. В този случай не се поставя връщана стойност след израз `return`. Типична ситуация е да се излезе от функцията в зависимост от определени условия:

```
def print_person(name, age):  
    if age > 120 or age < 1:  
        print("Invalid age")  
        return  
    print(f"Name: {name} Age: {age}")  
print_person("Tom", 22)  
print_person("Bob", -102)
```



```
main.py  
1 def print_person(name, age):  
2     if age > 120 or age < 1:  
3         print("Invalid age")  
4         return  
5     print(f"Name: {name} Age: {age}")  
6 print_person("Tom", 22)  
7 print_person("Bob", -102)
```

Run Shell

Name: Tom Age: 22
Invalid age
> |

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Тук функцията `print_person` приема `name` и `age` на потребителя като параметри. Във функцията, обаче първо се проверява дали възрастта е в определен диапазон (по-малко от 120 и по-голямо от 0). Ако възрастта е извън този диапазон, тогава се показва съобщение за невалидна възраст и се излиза от функцията с помощта на оператора `return`. След това функцията приключва работата си. Въпреки това, ако възрастта е правилна, се отпечатва потребителската информация в конзолата. Резултатът е:

Name: Tom Age: 22

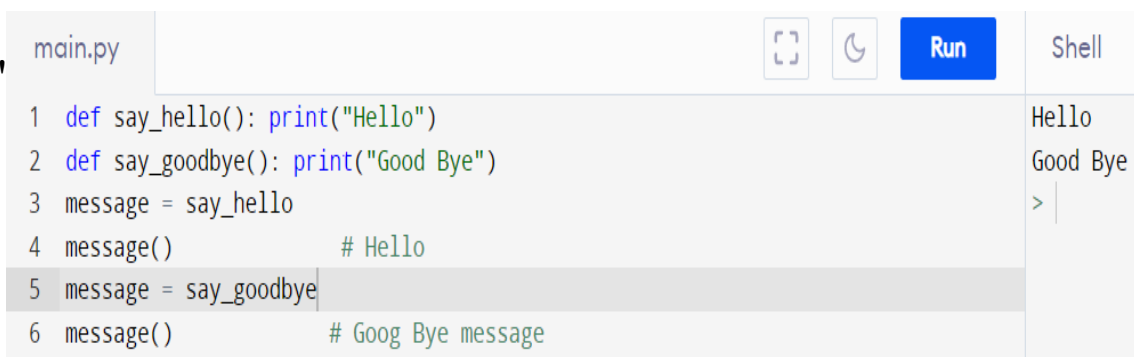
Invalid age

14. Функция като тип, параметър и резултат от друга функци

А. Функция като тип

В Python функцията всъщност представлява един тип. Така че може да се присвои функция на променлива и след това да се извика тази функция с помощта на променливата. Например:

```
def say_hello(): print("Hello")
def say_goodbye(): print("Good Bye")
message = say_hello
message()          # Hello
message = say_goodbye
message()          # Goog Bye
message
```



```
main.py
1 def say_hello(): print("Hello")
2 def say_goodbye(): print("Good Bye")
3 message = say_hello
4 message()          # Hello
5 message = say_goodbye
6 message()          # Goog Bye message
```

Run

Shell

Hello
Good Bye
> |

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

В този случай една от функциите се присвоява на променливата. Първо, на нея се предава функция `say_hello()`:

```
message = say_hello
```

след това променливата на съобщението ще сочи към тази функция, т.е. всъщност ще представлява функцията `say_hello`. Това означава, че може да се извика променливата със съобщението като обикновена функция:

```
message()    # Hello
```

Това ефективно ще изпълни функцията `say_hello` и ще отпечата низа "Hello" в конзолата. След това, по подобен начин, могат да се предаде друга функция на променливата на съобщението и да се извика.

По същия начин може да се извика функция с параметри чрез променлива и да се върне нейния резултат:

```
def sum(a, b): return a + b
def multiply(a, b): return a * b
operation = sum
result = operation(5, 6)
print(result)           # 11
operation = multiply
print(operation(5, 6))   # 30
```

main.py					Shell
1	<code>def sum(a, b): return a + b</code>				11
2	<code>def multiply(a, b): return a * b</code>				30
3	<code>operation = sum</code>				>
4	<code>result = operation(5, 6)</code>				
5	<code>print(result)</code>				# 11
6	<code>operation = multiply</code>				
7	<code>print(operation(5, 6))</code>				# 30

Б. Функция като функционален параметър

Тъй като функцията в Python може да представлява същата стойност като низ или число, може съответно да се предаде като параметър на друга функция. Например, нека се дефинира функция, която отпечатва резултата от някаква операция в конзолата:

```
def do_operation(a, b, operation):
```

```
    result = operation(a, b)
```

```
    print(f"result = {result}")
```

```
def sum(a, b): return a + b
```

```
def multiply(a, b): return a * b
```

```
do_operation(5, 4, sum)
```

```
do_operation(5, 4, multiply)
```

main.py

```
1 def do_operation(a, b, operation):
2     result = operation(a, b)
3     print(f"result = {result}")
4 def sum(a, b): return a + b
5 def multiply(a, b): return a * b
6 do_operation(5, 4, sum)           # result = 9
7 do_operation(5, 4, multiply)      # result = 20
```



Run

Shell

```
result = 9
result = 20
> |
```

В този случай функцията `do_operation` има три параметъра, като третият параметър се предполага, че е функция, която приема два параметъра и връща някакъв резултат. С други думи, третият параметър – `operation`, представлява някаква операция, но в момента на дефиниране на функцията `do_operation`, не се знае точно какъв вид операция е това. Знае се само, че функцията приема два параметъра и връща някакъв резултат, който след това се отпечатва в конзолата. Когато се извика функцията `do_operation`, може да се предаде друга функция, като трети параметър, например функцията `sum`:

```
do_operation(5, 4, sum)
```

В този случай параметърът на операцията представлява функцията `sum` и ще върне сбора от две числа. След това, по подобен начин, при извикването на функцията `do_operation`, може да се предаде друга функция, `multiply`, към третия параметър, който ще извърши умножението на числата:

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

```
do_operation(5, 4, multiply)      # result = 20
```

По този начин функции, които са по-гъвкави във функционалност, могат да приемат други функции чрез параметри.

В. Функция като резултат от функция

Една функция в Python може да върне друга функция. Например, ако се дефинира функция, която според стойността на параметъра връща една или друга операция:

```
def sum(a, b): return a + b
```

```
def subtract(a, b): return a - b
```

```
def multiply(a, b): return a * b
```

```
def select_operation(choice):
```

```
    if choice == 1:
```

```
        return sum
```

```
    elif choice == 2:
```

```
        return subtract
```

```
    else:
```

```
        return multiply
```

```
operation = select_operation(1)
```

```
print(operation(10, 6))
```

```
operation = select_operation(2)
```

```
print(operation(10, 6))
```

```
operation = select_operation(3)
```

```
print(operation(10, 6))
```

main.py	Run	Shell
1 def sum(a, b): return a + b		16
2 def subtract(a, b): return a - b		4
3 def multiply(a, b): return a * b		60
4 def select_operation(choice):		>
5 if choice == 1:		
6 return sum		
7 elif choice == 2:		
8 return subtract		
9 else:		
10 return multiply		
11 operation = select_operation(1)	# operation = sum	
12 print(operation(10, 6))	# 16	
13 operation = select_operation(2)	# operation = subtract	
14 print(operation(10, 6))	# 4	
15 operation = select_operation(3)	# operation = multiply	
16 print(operation(10, 6))	# 60	

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

В този случай функцията `select_operation`, в зависимост от стойността на параметъра за избор, връща една от трите функции - сумиране, изваждане и умножение. Така се получава резултат от функцията `select_operation` в променливата `operation`:

```
operation = select_operation(1)
```

Така, че в този случай числото 1 се предава на функцията `select_operation`, съответно тя ще върне функцията `sum`. Следователно операцията с променлива ще сочи към функцията сума, която извършва събирането на две числа:

```
print(operation(10, 6))          # 16 - фактически равно sum(10, 6)
```

По същия начин може да се получат и изпълнят и други функции.

15. Ламбда изрази

Ламбда изразите в Python са малки анонимни функции, които се дефинират с помощта на ламбда оператора. Формалната дефиниция на ламбда израз е:

lambda [параметри] : инструкция

Нека се дефинира най-простия ламбда израз:

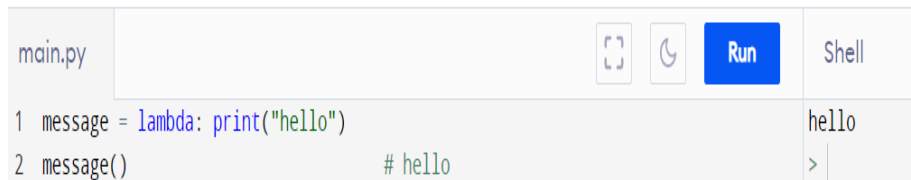
```
message = lambda: print("hello")
```

```
message()      # hello
```

Тук ламбда изразът се присвоява на променливата `message`. Този ламбда израз няма параметри, не връща нищо и просто отпечатва низа "hello" в конзолата. Чрез променливата `message` може да се нарече този ламбда израз, като нормална функция. Всъщност това е подобно на функцията:

```
def message():
```

```
    print("hello")
```



The screenshot shows a code editor with a file named `main.py`. The code contains two lines: `1 message = lambda: print("hello")` and `2 message()`. The second line has a green comment `# hello`. To the right of the code are icons for file operations and a blue `Run` button. On the far right, a `Shell` window displays the output `hello` followed by a prompt `>`.

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Ако ламбда изразът има параметри, дефиницията става чрез ключовата дума `lambda`. Ако ламбда изразът върне резултат, тогава той се посочва след двоеточие. Например, нека се дефинира ламбда израз, който връща квадрата на число:

```
square = lambda n: n * n
print(square (4))      #16
print (square (5))     #25
```

main.py		Run	Shell
1	<code>square = lambda n: n * n</code>		16
2	<code>print(square (4))</code>		25
3	<code>print (square (5))</code>		>

В този случай ламбда изразът приема един параметър - `n`. Вдясно от двоеточието е връщаната стойност - `n * n`. Този ламбда израз е подобен на слелната функция:

```
def square2(n): return n * n
print (square2 (5))
```

main.py		Run	Shell
1	<code>def square2(n): return n * n</code>		25
2	<code>print (square2 (5))</code>		>

По същия начин може да се създадат ламбда изрази, които приемат множество параметри:

```
sum = lambda a, b: a + b
print (sum(4, 5))      #9
print (sum(5, 6))     #11
```

main.py		Run	Shell
1	<code>sum = lambda a, b: a + b</code>		9
2	<code>print (sum(4, 5))</code>		11
3	<code>print (sum(5, 6))</code>		>

Докато ламбда изразите позволяват леко да съкратите дефинициите на функциите, те все още са ограничени, тъй като могат да изпълняват само един израз. Въпреки това, те могат да бъдат доста удобни в случаите, когато трябва да се използва функция, която да бъде предадена като параметър или върната към друга функция. Например, подаване на ламбда израз като параметър:

```
def do_operation(a, b, operation):
    result = operation (a, b)
    print(f'result = {result}')
do_operation(5, 4, lambda a, b: a + b)
do_operation(5, 4, lambda a, b: a * b)
```

main.py		Run	Shell
1	<code>def do_operation(a, b, operation):</code>		result = 9
2	<code> result = operation (a, b)</code>		result = 20
3	<code> print(f'result = {result}')</code>		>
4	<code>do_operation(5, 4, lambda a, b: a + b)</code>	# result = 9	
5	<code>do_operation(5, 4, lambda a, b: a * b)</code>	# result = 20	

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

В този случай не е необходимо да се дефинират функции, за да бъдат предадени като параметър.

Същото важи и за връщането на ламбда изрази от функции:

```
def select_operation(choice):  
    if choice == 1:  
        return lambda a, b: a + b  
    elif choice == 2:  
        return lambda a, b: a - b  
    else:  
        return lambda a, b: a * b  
operation = select_operation(1)  
print(operation (10, 6))  
operation = select_operation(2)  
print(operation (10, 6))  
operation = select_operation(3)  
print(operation (10, 6))
```

main.py	Run	Shell
1 def select_operation(choice):		16
2 if choice == 1:		4
3 return lambda a, b: a + b		60
4 elif choice == 2:		>
5 return lambda a, b: a - b		
6 else:		
7 return lambda a, b: a * b		
8 operation = select_operation(1)	# operation = sum	
9 print(operation (10, 6))	#16	
10 operation = select_operation(2)	# операция = subtract	
11 print(operation (10, 6))	#4	
12 operation = select_operation(3)	# операция = multiply	
13 print(operation (10, 6))	#60	

16. Преобразуване на типа

Стойности от различни типове могат да се използват в операции с данни. Например, добавяне на число от тип int и число от тип float:

main.py	Run	Shell
1 a = 2		4.5
2 b = 2.5		>
3 c = a + b		
4 print(c)		

В този случай няма да има грешка. Въпреки това е възможно Python не винаги да изпълнява автоматично операции, които включват данни от различен тип. Нека видим какви правила се прилагат в този случай.

17. Неявни преобразувания

И двете числа в аритметичните операции трябва да представляват един и същи тип. Ако двата операнда на операцията представляват различни типове данни, тогава Python се опитва автоматично да преобразува в един от типовете съгласно следните правила:

- Ако един от операндите на операцията е комплексно число, тогава другият операнд също се преобразува в комплексен тип.
- В противен случай, ако един от операндите е от тип `float`, тогава вторият операнд също се преобразува в тип `float`. Всъщност това се случи в примера от предходния слайд, където стойността на променливата `a` беше преобразувана в типа `float`.
- В противен случай и двата операнда трябва да са от тип `int`, като в този случай не се изисква преобразуване.

18. Изрични преобразувания

Но в някои случаи става необходимо ръчно преобразуване на типа. Ако се разполага със следния код:

```
a = "2"  
b = 3  
c = a + b
```

Очаква се `"2" + 3` да е равно на 5. Този код обаче ще предизвика изключение, защото първото число всъщност представлява низ. При изпълнение на кода ще се изведе грешка:

Traceback (most recent call last):

File `"/Users/eugene/PycharmProjects/HelloApp/main.py"`, line 3, in

```
c = a + b
```

`TypeError: can only concatenate str (not "int") to str`

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Python предоставя редица вградени функции за преобразуване на типове:

- `int()` : преобразува стойността в цяло число
- `float()` : преобразува стойност в `float`
- `str()` : преобразува стойността в низ

18.1. `int`

И така, в предишния пример става преобразуване на низ в число, като се използва функцията

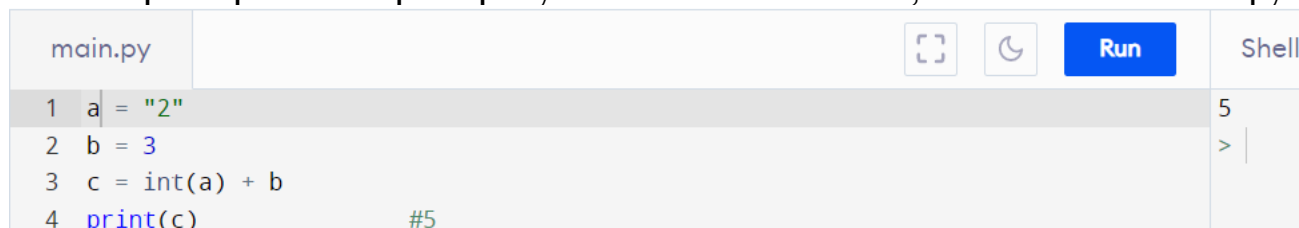
`int()`:

`a = "2"`

`b = 3`

`c = int(a) + b`

`print(c)` #5



```
main.py
1 a = "2"
2 b = 3
3 c = int(a) + b
4 print(c) #5
```

5

Примери за трансформации с помощта на `int()`:

`a = int(15)` # `a = 15`

`b = int(3.7)` # `b = 3`

`c = int("4")` # `c = 4`

`e = int(False)` # `e = 0`

`f = int(True)` # `f = 1`

Ако обаче стойността не може да бъде преобразувана, тогава функцията `int` ще изведе грешка `ValueError: invalid literal for int() with base 10:`

`b = int("a1c")` # Error

`c = int("4.7")` # Error

18.2. float

Функцията `float()` работи по подобен начин , като се преобразува в число с плаваща запетая.

```
a = "2,7"
b = 3
c = float(a) + b
print(c)
```

5.7

main.py

```
1 a = "2.7"
2 b = 3
3 c = float(a) + b
4 print(c)
```

5.7



Run

Shell

5.7

> |

Примери за трансформации с помощта на `float()`:

```
a = float(15)      # a = 15.0
b = float(3.7)     # b = 3.7
c = float("4.7")   # c = 4.7
d = float("5")     # d = 5.0
e = float(False)   # e = 0.0
f = float(True)    # f = 1.0
```

Но отново, не всички стойности могат да бъдат автоматично преобразувани във `float`. Така че в следния случай Python ще генерира грешка:

```
d = float("abc")    # Error
```

18.3. str

Функцията `str()` преобразува стойност в низ:

```
a = str(False)     # a = "False"
b = str(True)      # b = "True"
c = str(5)         # c = "5"
d = str(5.7)       # d = "5.7"
```

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Функцията `str()` може да бъде уместна, например, когато се добавя стойност от друг тип към низ. **Например в следния случай ще получим грешка:**

```
age=22
message = "Age: " + age      # Error
print (message)
```

Ако към число се добави число, това е стандартна операция за събиране на числа. Ако низ е добавен към низ, тогава това е операция за конкатенация на низове. Но Python не знае как да извърши операцията за събиране по отношение на низ и число. И ако в този случай трябва да се изпълни операция за конкатениране на низове, тогава числото може да бъде намалено до низ с помощта на функцията **`str()`**:

```
age=22
message = "Age: " + str(age)
print (message)
```

main.py	  	Shell
1 age=22		Age: 22
2 message = "Age: " + str(age)	# Age: 22	>
3 print (message)		

19. Променлив обхват

Обхватът определя контекста на променлива, в който може да се използва. Има два типа контекст в Python: глобален и локален.

19.1. Глобален контекст

Глобалният контекст предполага, че променливата е глобална, дефинирана е извън, която и да е от функциите и е достъпна за всяка функция в програмата. Например:

```
name = "Tom"
```

```
def say_hi():  
    print("Hello", name)  
def say_bye():  
    print("Good bye", name)  
say_hi()  
say_bye()
```

main.py	Run	Shell
<pre>1 name = "Tom" 2 def say_hi(): 3 print("Hello", name) 4 def say_bye(): 5 print("Good bye", name) 6 say_hi() 7 say_bye()</pre>		<pre>Hello Tom Good bye Tom > </pre>

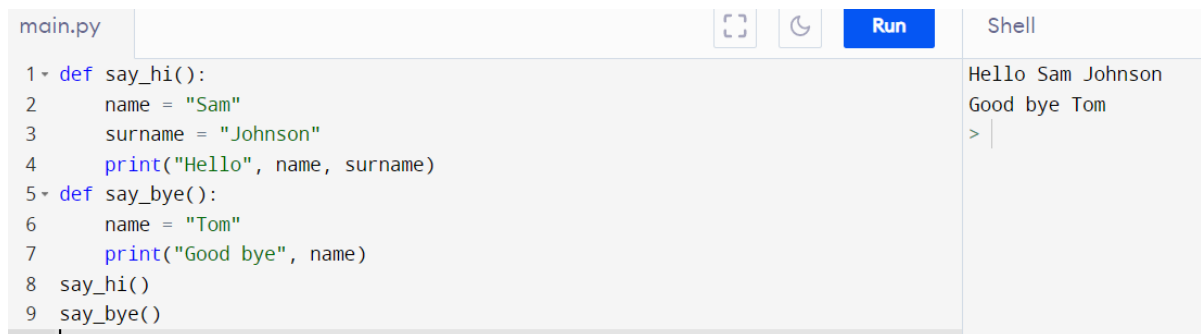
Тук променливата `name` е глобална и има глобален обхват.

19.2. Локален контекст

За разлика от глобалните променливи, локалната променлива се дефинира вътре във функция и е достъпна само от тази функция, т.е. има локален обхват:

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

```
def say_hi():  
    name = "Sam"  
    surname = "Johnson"  
    print("Hello", name, surname)  
def say_bye():  
    name = "Tom"  
    print("Good bye", name)  
say_hi()  
say_bye()
```



```
main.py  Run  Shell  
1 def say_hi():  
2     name = "Sam"  
3     surname = "Johnson"  
4     print("Hello", name, surname)  
5 def say_bye():  
6     name = "Tom"  
7     print("Good bye", name)  
8 say_hi()  
9 say_bye()  
10  
Hello Sam Johnson  
Good bye Tom  
> |
```

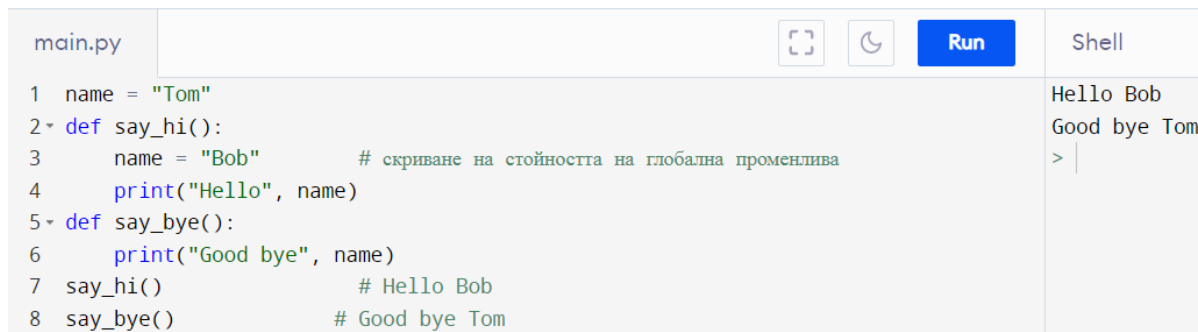
В този случай във всяка от двете функции се дефинира `name` на локална променлива. И въпреки че тези променливи са наречени еднакво, те все пак са две различни променливи, всяка от които е достъпна само в рамките на собствената си функция. Също така, във функцията `say_hi()` е дефинирана променливата `surname`, която също е локална, така, че функцията `say_bye()` не може да я използва.

19.3. Скриване на променливи

Има и друга опция за дефиниране на променлива, когато локална променлива крие глобална със същото име:

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

```
name = "Tom"
def say_hi():
    name = "Bob"      # скриване на стойността на глобална променлива
    print("Hello", name)
def say_bye():
    print("Good bye", name)
say_hi()              # Hello Bob
say_bye()             # Good bye Tom
```



```
main.py
1 name = "Tom"
2 def say_hi():
3     name = "Bob"      # скриване на стойността на глобална променлива
4     print("Hello", name)
5 def say_bye():
6     print("Good bye", name)
7 say_hi()              # Hello Bob
8 say_bye()             # Good bye Tom

Run Shell
Hello Bob
Good bye Tom
> |
```

В конкретния случай е дефинирано името на глобалната променлива. Въпреки това, локална променлива със същото име е дефинирана във функцията `say_hi`. И ако функцията `say_bye` използва глобална променлива, тогава функцията `say_hi` използва локална променлива, която скрива глобалната.

Ако се налага промяна на глобалната променлива в локална функция, а не да се дефинира локална, тогава трябва да се използва ключова дума `global`:

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

```
name = "Tom"
def say_hi():
    global name
    name = "Bob"
    print("Hello", name)
def say_bye():
    print("Good bye", name)
say_hi()
say_bye()
```

main.py	Run	Shell
<pre>1 name = "Tom" 2 def say_hi(): 3 global name 4 name = "Bob" # изменение на глобалната променлива 5 print("Hello", name) 6 def say_bye(): 7 print("Good bye", name) 8 say_hi() # Hello Bob 9 say_bye() # Good bye Bob</pre>		<pre>Hello Bob Good bye Bob > </pre>

19.4. nonlocal (нелокален)

Нелокалният израз прикрепя идентификатор към променлива от най-близкия заобикалящ контекст (с изключение на глобалния контекст). Обикновено `nonlocal` се използва във вложени функции, когато трябва да се прикачи идентификатор зад променлива или параметър на заобикалящата външна функция. Изразът демонстрира подобна ситуация:

```
def outer():
    n = 5
    def inner():
        print(n)
    inner() # 5
    print(n)
outer() # 5
```

main.py	Run	Shell
<pre>1 def outer(): # външна функция 2 n = 5 3 def inner(): # вложена функция 4 print(n) 5 inner() # 5 6 print(n) 7 outer() # 5</pre>		<pre>5 5 > </pre>

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Тук вложената локална функция `inner()` отпечатва в конзолата стойността на променливата `n`, която е дефинирана във външната функция `outer()`. След това функцията `outer()` извиква вътрешната функция `inner()`. Когато се извика функцията `outer()` тук, е възможно да видим числото 5 да се появи два пъти в конзолата. Въпреки това, в този случай, вложената функция `inner()` просто получава стойността. Сега да разгледаме друга ситуация, при която вложена функция присвоява стойност на променлива:

```
def outer():  
    n = 5  
    def inner():  
        n = 25  
        print(n)  
    inner()  
    print(n)  
outer()  
# 25 - inner  
# 5 - outer
```

main.py	Run	Shell
1 def outer(): # външна функция		25
2 n = 5		5
3 def inner(): # вложена функция		>
4 n = 25		
5 print(n)		
6 inner() # 25		
7 print(n)		
8 outer() # 5		
9 # 25 - inner		
10 # 5 - outer		

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

При присвояване на стойност във вложена функция: $n = 25$ ще се създаде нова променлива n , която ще скрие променливата n от външната функция. В резултат на това ще се получат две различни числа при извеждане.

За да посочите във вложена функция, че идентификаторът във вложената функция ще представлява променлива от заобикалящата функция, се използва нелокалният израз:

```
def outer():  
    n=5  
    def inner():  
        nonlocal n  
        n=25  
        print(n)  
    inner()  
    print (n)  
outer()
```

main.py		Run	Shell
1	def outer():	# външна функция	25
2	n=5		25
3	def inner():	# вложена функция	>
4	nonlocal n	# показва, че n е променлива от заобикалящата функция	
5	n=25		
6	print(n)		
7	inner()	#25	
8	print (n)		
9	outer()	#25	

20. Затваряния

Затварянето е функция, която помни своята лексикална среда, дори когато се изпълнява извън нейния обхват.

Технически, затварянето има три компонента:

- външна функция, която дефинира някакъв обхват и в която са дефинирани някои променливи и параметри - лексикална среда;
- променливи и параметри (лексикална среда), които са дефинирани във външната функция;
- вложена функция, която използва променливите и параметрите на външната функция.

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Python използва локални функции за дефиниране на затваряния:

```
def outer():  
    n = 5  
    def inner():  
        nonlocal n  
        n += 1  
        print(n)  
    return inner  
fn = outer()
```

```
fn()  
fn()  
fn()
```

main.py		Run	Shell
1 def outer():	# външна функция		6
2 n = 5	# лексикална среда - локална променлива		7
3 def inner():	# локална функция		8
4 nonlocal n			>
5 n += 1	# операции за лексикална среда		
6 print(n)			
7 return inner			
8 fn = outer()	# fn = inner, тъй като външната функция връща		
	вътрешната функция		
9 # извикване на вътрешна функция			
10 fn()	#6		
11 fn()	#7		
12 fn()	#8		

Тук функцията `outer` дефинира локална променлива `n` - това е лексикалната среда за вътрешната функция. Вътре във външната функция е дефинирана вътрешна функция - вътрешната локална функция, която осъществява достъп до своята лексикална среда - променливата `n` - увеличава стойността си с 1 и извежда резултата в конзолата:

```
def inner():  
    # локална функция  
    nonlocal n  
    n += 1  
    # операции за лексикална среда  
    print(n)
```

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Тази локална функция се връща от външната функция:

```
return inner
```

В програмата се извиква външната функция и се получава `fn` локална вътрешна функция в променлива:

```
fn = outer()
```

Променливата `fn` е затваряне, т.е. комбинира две неща: функция и средата, в която е създадена функцията. И въпреки факта, че се получава локална функция и може да се извика извън нейната заобикаляща функция, в която е дефинирана, въпреки това тя запомня своята лексикална среда и може да има достъп и да се променя, което се вижда от изхода на конзолата:

```
fn() # 6
```

```
fn() # 7
```

```
fn() # 8
```

21. Прилагане на опции

Освен външни променливи, лексикалната среда включва и параметри на заобикалящата функция. Помислете за използването на параметри:

```
def multiply(n):
```

```
    def inner(m): return n * m
```

```
    return inner
```

```
fn = multiply(5)
```

```
print(fn(5))
```

```
print(fn(6))
```

```
print(fn(7))
```

main.py

```
1 def multiply(n):
2     def inner(m): return n * m
3     return inner
4 fn = multiply(5)
5 print(fn(5))      # 25
6 print(fn(6))      # 30
7 print(fn(7))      # 35
```



Run

Shell

```
25
30
35
>
```

Тема 3_2. Цикли. Функции. Ламбда изрази. Преобразуване на тип. Обхват на променливите

Тук външната функция - multiply връща функция, която приема число и връща число.

Извикването на функцията multiply() връща локална вътрешна функция:

```
def inner(m): return n * m
```

Тази функция запомня средата, в която е създадена, по-специално стойността на параметъра n. Освен това самият той приема параметър и връща произведението на параметрите n и m. В резултат на това при извикване на функцията multiply се дефинира променлива fn, която получава вътрешната локална функция и нейната лексикална среда - стойността на параметъра n:

```
fn = multiply(5)
```

В този случай параметърът n е 5. При извикване на локална функция, например, в случай на:

```
print(fn(6))          # 30
```

Числото 6 се предава като параметър m на локална функция, която връща произведението на n и m, т.е. $5 * 6 = 30$. Също така би било възможно да се съкрати този код с ламбда израз:

```
def multiply(n): return lambda m: n * m
```

```
fn = multiply (5)
```

```
print(fn(5))
```

```
print(fn(6))
```

```
print(fn(7))
```

main.py			Shell
1	def multiply(n): return lambda m: n * m		25
2	fn = multiply (5)		30
3	print(fn(5)) #25		35
4	print(fn(6)) #30		>
5	print(fn(7)) #35		

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009